37

硬盘驱动器

上一章介绍了 I/O 设备的一般概念，并展示了操作系统可能会怎样和这样的“怪兽”打交道。在这一章，我们更深入地讨论一种具体的设备：**硬盘驱动器**（hard disk drive）。几十年来，这种驱动器一直是计算机系统中持久数据存储的主要形式，文件系统（file system）技术（马上就会讲到）的很多发展也都是以它的行为为前提的。因此，在构建管理磁盘的文件系统软件之前，有必要先了解磁盘的工作细节。这些细节有很多可以在 Ruemmler 和 Wilkes [RW92] 以及 Anderson、Dykes 和 Riedel [ADR03] 的优秀论文中找到。

关键问题：如何在磁盘上存储和访问数据
现代硬盘驱动器是怎样存储数据的？它的接口是什么样的？数据实际上是怎样布局和访问的？磁盘调度怎样提高性能？

37.1 接口

我们先来了解现代磁盘驱动器（disk drive）的接口。所有现代驱动器的基本接口都很简单。驱动器由大量扇区（sector，即 512 字节的块）组成，每个扇区都可以读或写。在一个有 n 个扇区的磁盘上，扇区编号从 0 到 $n-1$。因此，我们可以把磁盘看成一个扇区数组；0 到 $n-1$ 就是驱动器的**地址空间**（address space）。

多扇区操作（multi-sector operation）也是可以的，许多文件系统一次就会读写 4 KB（或者更多）。然而，在更新磁盘时，驱动器厂商唯一保证的是单次 512 字节的写入是**原子的**（atomic），也就是说，它要么完整地完成，要么根本不完成；因此，如果发生意外断电（power loss），较大的写入可能只完成了一部分（有时称为**撕裂写**，torn write）。

大多数磁盘驱动器的使用者都会做一些假设，但这些假设并没有直接写在接口里；Schlosser 和 Ganger 把它称为磁盘驱动器的“不成文的契约” [SG04]。具体来说，通常可以假设：在驱动器的地址空间中，访问两个相距较近的块（block）[1]要比访问两个相距很远的块快。通常还可以假设：访问一段连续的块（即顺序读或顺序写）是最快的访问方式，一般比任何更随机的访问模式快得多。

[1]我们（以及其他人）常常把**块**（block）和**扇区**（sector）这两个词混着用，指望读者能根据上下文明白到底指什么。为此说声抱歉！

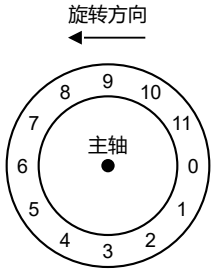


图 37.1：**只有一个磁道的磁盘**

37.2 基本几何结构

下面来认识现代磁盘的几个组成部分。先从**盘片**（platter）说起，它是一个圆形的硬质表面，靠在上面引起磁性变化来持久地存储数据。一个磁盘可以有一个或多个盘片；每个盘片有两面，每一面称为一个**盘面**（surface）。这些盘片通常由某种坚硬的材料（比如铝）制成，上面再涂一层很薄的磁性层，让驱动器即使断电也能持久地保存这些位。

所有盘片都绑在**主轴**（spindle）上，主轴连着一个电机，驱动器通电时，电机以恒定（固定）的速率带动盘片旋转。转速通常用**每分钟转数**（rotations per minute，RPM）来衡量，现代驱动器的典型值在 7,200 RPM 到 15,000 RPM 之间。注意，我们常常关心的是转一圈所用的时间，例如，转速为 10,000 RPM 的驱动器转一圈大约需要 6 毫秒（6 ms）。

每个盘面上，数据编码在一圈圈由扇区组成的同心圆上；这样的一圈称为一条**磁道**（track）。一个盘面上有成千上万条磁道，排列得非常紧密，一根头发丝的宽度就能容纳几百条磁道。

要在盘面上读写，我们需要一种机制（mechanism），既能感知（即读取）磁盘上的磁性图案，也能引起（即写入）磁性变化。读写这一过程由**磁头**（disk head）完成；驱动器的每个盘面都有一个这样的磁头。磁头安装在一个**磁盘臂**（disk arm）上，磁盘臂在盘面上移动，把磁头定位到所需的磁道上方。

37.3 一个简单的磁盘驱动器

我们一次加一条磁道，逐步搭起一个模型，以此来理解磁盘是怎样工作的。假设我们有一个只有一条磁道的简单磁盘（图 37.1）。这条磁道只有 12 个扇区，每个扇区 512 字节（回想一下，这是我们常用的扇区大小），因此用 0 到 11 来编址。这里只有一个盘片，它绕着主轴（spindle）旋转，主轴上连着一个电机。

当然，单单一条磁道没什么意思；我们希望能读写这些扇区，因此需要一个磁头，装在磁盘臂上，就像现在看到的这样（图 37.2）。在图中，装在磁盘臂末端的磁头位于扇区 6 上方，盘面按逆时针方向旋转。

 © 2008–23, ARPACI-DUSSEAU

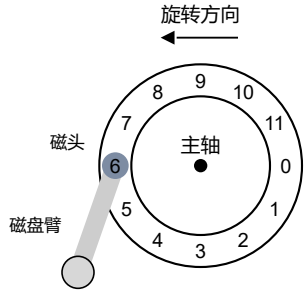


图 37.2：**单个磁道加一个磁头**

单磁道延迟：旋转延迟

为了理解在这个简单的单磁道磁盘上请求是怎样处理的，设想我们现在收到一个读取块 0 的请求。磁盘该怎样处理这个请求呢?

在我们这个简单的磁盘上，磁盘不需要做多少事。具体来说，它只要等待所需的扇区转到磁头下方就行了。在现代驱动器中，这种等待很常见，又是 I/O 服务时间中一个相当重要的组成部分，所以它有一个专门的名字：**旋转延迟**（rotational delay，有时也叫 **rotation delay**，不过这听起来有点怪）。在这个例子中，如果转一整圈的延迟是 R，磁头又是从扇区 6 开始的，那么磁盘大约要等 $\frac{R}{2}$ 的旋转延迟，扇区 0 才会转到读写磁头（read/write head）下方。在这条磁道上，最坏的请求是读扇区 5，处理这样的请求，几乎要等整整一圈的旋转延迟。

多磁道：寻道时间

到目前为止，我们的磁盘只有一条磁道，这不太现实；现代磁盘当然有几百万条磁道。所以，我们来看一个稍微更现实一点的盘面，它有三条磁道，见图 37.3 左。

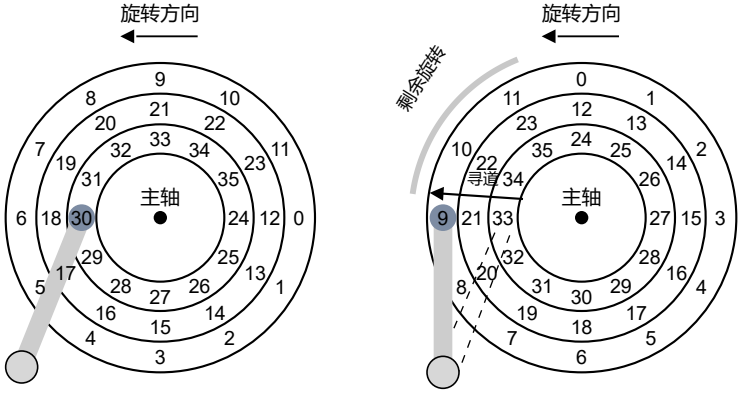


图 37.3：**三个磁道加一个磁头（右：带寻道）**

 © 2008–23, ARPACI-DUSSEAU

在图中，磁头当前位于最内侧的磁道上方，这条磁道包含扇区 24 到 35；往外一条磁道包含接下来的一组扇区 12 到 23，最外侧的磁道包含最前面的扇区 0 到 11。

为了理解驱动器怎样访问某个扇区，我们跟踪一个对较远扇区的请求（例如读扇区 11），看看会发生什么。为了处理这次读，驱动器必须先把磁盘臂移到正确的磁道（这里是最外侧的磁道），这个过程叫作**寻道**（seek）。寻道和旋转一样，是代价最高的磁盘操作之一。

需要指出的是，寻道分好几个阶段：首先是磁盘臂开始移动的加速（acceleration）阶段；然后是磁盘臂全速移动的匀速滑行（coasting）阶段；接着是磁盘臂减慢下来的减速（deceleration）阶段；最后是把磁头小心地停在正确磁道上方的 *settling* 阶段。**settling time** 往往相当长，例如 0.5 到 2 毫秒，因为驱动器必须确保找到了正确的磁道——想象一下，如果它只是停在附近会怎样！

寻道结束后，磁盘臂已经把磁头定位到了正确的磁道上方。寻道的示意见图 37.3（右）。

可以看到，在寻道期间，磁盘臂移到了目标磁道，而盘片当然也转动了，这里大约转过了 3 个扇区。因此，扇区 9 正要从磁头下方经过，我们只需再等一小段旋转延迟就能完成传输。

扇区 11 经过磁头下方时，I/O 的最后一个阶段就开始了，这个阶段叫作**传输**（transfer），数据在这时从盘面读出或写入盘面。这样，我们就得到了 I/O 时间的完整图景：先寻道，再等待旋转延迟，最后传输。

其他一些细节

虽然我们不会在这上面花太多时间，但关于硬盘的工作方式，还有一些其他有意思的细节。许多驱动器采用某种**磁道偏斜**（track skew），确保即使跨越磁道边界，顺序读也能顺利完成。在我们这个简单的示例磁盘上，它可能是图 37.4（第 4 页）的样子。

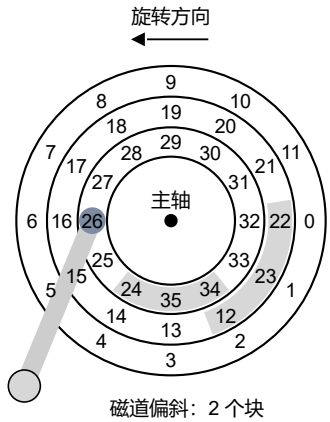


图 37.4：**三个磁道：磁道偏斜为 2**

扇区之所以常常像这样偏斜，是因为从一条磁道切换到另一条磁道时，即使是相邻的磁道，磁盘也需要时间重新定位磁头。如果没有这种偏斜，磁头移到下一条磁道时，所需的下一个块已经转过了磁头，驱动器就得等上几乎一整圈的旋转延迟才能访问下一个块。

另一个现实是，外侧磁道往往比内侧磁道有更多的扇区，这是几何结构造成的；外面的空间就是更大。这类驱动器通常称为 **multi-zoned** 磁盘驱动器：磁盘被组织成多个区（zone），一个区就是一个盘面上一组连续的磁道。同一个区内每条磁道的扇区数相同，外侧的区比内侧的区扇区更多。

最后，任何现代磁盘驱动器都有一个重要组成部分：**缓存**（cache），由于历史原因，有时也叫**磁道缓冲区**（track buffer）。这个缓存就是少量内存（通常大约 8 MB 或 16 MB），驱动器可以用它保存从磁盘读出或要写入磁盘的数据。例如，从磁盘读一个扇区时，驱动器可能会决定把这条磁道上的所有扇区都读进来，缓存在它的内存里；这样，后续对同一磁道的任何请求，驱动器都能很快响应。

对于写操作，驱动器有一个选择：是在把数据放进自己的内存时就确认（acknowledge）写入已完成，还是在数据真正写到磁盘上之后再确认？前者称为**回写**（write back）缓存（有时也叫 **immediate reporting**），后者称为**直写**（write through）。回写缓存（write-back caching）有时会让驱动器显得“更快”，但也可能很危险：如果文件系统或应用程序为了正确性要求数据按一定顺序写到磁盘上，回写缓存就可能带来问题（详见关于文件系统日志（journaling）的那一章）。

37.4 I/O 时间：算一算

现在我们有了磁盘的抽象模型，可以做一点分析，更好地理解磁盘的性能。具体来说，我们现在可以把 I/O 时间表示为三个主要组成部分之和：

$$T_{I/O} = T_{seek} + T_{rotation} + T_{transfer} \tag{37.1}$$

注意，I/O 速率（$R_{I/O}$）往往更便于比较不同的驱动器（我们下面就会这样做），它很容易由时间算出来。只要用传输的大小除以所用的时间：

$$R_{I/O} = \frac{Size_{Transfer}}{T_{I/O}} \tag{37.2}$$

为了对 I/O 时间有更直观的感受，我们来做下面的计算。假设我们关心两种工作负载（workload）。第一种叫**随机**（random）工作负载，它向磁盘上的随机位置发出小的（例如 4 KB）读请求。随机工作负载在许多重要的应用中都很常见，包括数据库管理系统（database management system）。第二种叫**顺序**（sequential）工作负载，它只是从磁盘上连续读取大量扇区，不跳来跳去。顺序访问模式也很常见，因此同样重要。

为了理解随机和顺序两种工作负载在性能上的差别，我们需要先对磁盘驱动器做一些假设。我们来看希捷（Seagate）的两款现代磁盘。第一款是 Cheetah 15K.5 [S09b]，一款高性能的 SCSI 驱动器。第二款是 Barracuda [S09a]，一款为容量而打造的驱动器。两者的详细参数见图 37.5。

补充：量纲分析

还记得化学课上你是怎么解几乎每一道题的吗？只要把单位摆好，让它们互相约掉，答案似乎就自己冒出来了。这种化学魔法有个高大上的名字，叫作**量纲分析**（dimensional analysis），而它在计算机系统分析中其实也很有用。

我们来看一个例子，看看量纲分析（dimensional analysis）是怎样工作的，又为什么有用。假设你要算出磁盘转一圈需要多少毫秒。不巧的是，你只知道磁盘的 **RPM**，即**每分钟转数**（rotations per minute）。假设我们说的是一块 10K RPM 的磁盘（即每分钟转 10,000 圈）。怎样列出量纲分析（dimensional analysis）的式子，才能得到以毫秒为单位的每圈时间呢？

为此，我们先把想要的单位写在左边；这里我们想得到每转一圈的时间（以毫秒计），所以就这样写：$\frac{\text{时间 (ms)}}{\text{1 圈}}$。然后写下我们知道的所有东西，并尽可能让单位约掉。首先是 $\frac{\text{1 分钟}}{\text{10,000 圈}}$（把“圈”留在分母上，因为左边它就在分母上），然后用 $\frac{\text{60 秒}}{\text{1 分钟}}$ 把分钟换成秒，最后用 $\frac{\text{1000 ms}}{\text{1 秒}}$ 把秒换成毫秒。最终结果如下（单位都漂亮地约掉了）：

$$\frac{\text{时间 (ms)}}{\text{1 圈}} = \frac{1\cancel{\text{分钟}}}{\text{10,000 圈}} \cdot \frac{60\cancel{\text{秒}}}{1\cancel{\text{分钟}}} \cdot \frac{\text{1000 ms}}{1\cancel{\text{秒}}} = \frac{\text{60,000 ms}}{\text{10,000 圈}} = \frac{\text{6 ms}}{\text{圈}}$$

从这个例子可以看出，量纲分析（dimensional analysis）把看似凭直觉的事情变成了一个简单、可重复的过程。除了上面的 RPM 计算，它在 I/O 分析中也经常派得上用场。例如，题目常常会告诉你磁盘的传输速率，比如 100 MB/秒，然后问你：传输一个 512 KB 的块要多少毫秒？用量纲分析（dimensional analysis），这很容易：

$$\frac{\text{时间 (ms)}}{\text{1 次请求}} = \frac{512\cancel{\text{KB}}}{\text{1 次请求}} \cdot \frac{1\cancel{\text{MB}}}{1024\cancel{\text{KB}}} \cdot \frac{1\cancel{\text{秒}}}{100\cancel{\text{MB}}} \cdot \frac{\text{1000 ms}}{1\cancel{\text{秒}}} = \frac{\text{5 ms}}{\text{请求}}$$

可以看到，这两款驱动器的特性大不相同，在很多方面很好地概括了磁盘驱动器市场的两个重要组成部分。第一个是“高性能”驱动器市场，这类驱动器的设计目标是转速尽可能快、寻道时间（seek time）尽可能短、传输数据尽可能快。第二个是“容量”市场，这里每字节的成本最重要；因此，这类驱动器更慢，但会在可用空间里塞进尽可能多的位。

根据这些数字，我们可以开始计算这两款驱动器在上面描述的两种工作负载下表现如何。先来看随机工作负载。假设每次 4 KB 的读都发生在磁盘上的随机位置，我们可以算出每次这样的读要花多长时间。在 Cheetah 上：

	Cheetah 15K.5	Barracuda
容量	300 GB	1 TB
转速（RPM）	15,000	7,200
平均寻道时间	4 ms	9 ms
最大传输速率	125 MB/s	105 MB/s
盘片数	4	4
缓存	16 MB	16/32 MB
接口	SCSI	SATA

图 37.5：**磁盘驱动器规格：SCSI 与 SATA**

 © 2008–23, ARPACI-DUSSEAU

提示：顺序地使用磁盘
只要有可能，就顺序地在磁盘上读写数据。如果做不到，至少也要考虑按大块传输数据：越大越好。如果 I/O 以零碎的小块随机进行，I/O 性能会急剧下降。用户也会跟着受罪。你自己也会受罪，因为你知道，这些罪都是你那些粗心的随机 I/O 造成的。

$$T_{seek} = 4\ ms, T_{rotation} = 2\ ms, T_{transfer} = 30\ microsecs \tag{37.3}$$

平均寻道时间（4 毫秒）直接取的是厂商公布的平均时间；注意，从盘面一端到另一端的完整寻道可能要花两到三倍的时间。平均旋转延迟可以直接由 RPM 算出来。15000 RPM 等于 250 RPS（每秒转数）；因此，转一圈需要 4 ms。平均来说，磁盘要转半圈，所以平均时间是 2 ms。最后，传输时间（transfer time）就是传输大小除以峰值传输速率；这里它小得几乎可以忽略（30 微秒；注意，1000 微秒才是 1 毫秒！）。

因此，根据上面的公式，Cheetah 的 $T_{I/O}$ 大约是 6 ms。要计算 I/O 速率，只要用传输大小除以平均时间，就得到 Cheetah 在随机工作负载下的 $R_{I/O}$ 大约是 0.66 MB/s。对 Barracuda 做同样的计算，得到的 $T_{I/O}$ 大约是 13.2 ms，慢了一倍多，因此速率大约是 0.31 MB/s。

现在来看顺序工作负载。这里可以假设，在一次很长的传输之前只有一次寻道和一次旋转。为简单起见，假设传输大小是 100 MB。那么，Cheetah 和 Barracuda 的 $T_{I/O}$ 分别大约是 800 ms 和 950 ms。因此，I/O 速率分别非常接近 125 MB/s 和 105 MB/s 的峰值传输速率。图 37.6 汇总了这些数字。

这张图告诉我们几件重要的事。第一，也是最重要的一点：随机工作负载和顺序工作负载之间的驱动器性能差距巨大，Cheetah 相差将近 200 倍，Barracuda 相差 300 倍以上。于是我们得到了计算机历史上最显而易见的设计提示。

第二点更微妙一些：高端的“性能”驱动器和低端的“容量”驱动器之间，性能差距很大。由于这个原因（还有其他原因），人们常常愿意花大价钱买前者，而对后者则尽量买得便宜。

37.5 磁盘调度

由于 I/O 的开销很高，操作系统历来都会参与决定以什么顺序把 I/O 发给磁盘。更具体地说，给定一组 I/O 请求，**磁盘调度程序**（disk scheduler）会检查这些请求，决定下一个调度哪一个 [SCO90, JW91]。

作业调度通常不知道每个作业有多长；而磁盘调度不同，我们能比较准确地估计一个“作业”（即磁盘请求）要花多长时间。估计出一个请求的寻道时间和可能的旋转延迟，磁盘调度程序就能知道每个请求要花多长时间，从而贪心地（greedily）先选出处理时间最短

	Cheetah	Barracuda
$R_{I/O}$ 随机	0.66 MB/s	0.31 MB/s
$R_{I/O}$ 顺序	125 MB/s	105 MB/s

图 37.6：**磁盘驱动器性能：SCSI 与 SATA**

 © 2008–23, ARPACI-DUSSEAU

补充：“平均”寻道时间的计算

在许多书和论文中，你会看到平均磁盘寻道时间大约是完整寻道时间的三分之一。这个说法是从哪儿来的?

原来，它来自一个基于平均寻道距离（而不是时间）的简单计算。把磁盘想象成一组磁道，编号从 0 到 N。任意两条磁道 x 和 y 之间的寻道距离，就是它们之差的绝对值：$|x-y|$。

要计算平均寻道距离，只需先把所有可能的寻道距离加起来：

$$\sum_{x=0}^{N}\sum_{y=0}^{N}|x-y| \ . \tag{37.4}$$

然后，再除以不同寻道的总数：N^2。为了求和，我们直接用积分形式：

$$\int_{x=0}^{N}\int_{y=0}^{N}|x-y|\,\mathrm{d}y\,\mathrm{d}x. \tag{37.5}$$

为了计算内层积分，我们把绝对值拆开：

$$\int_{y=0}^{x}(x-y)\,\mathrm{d}y+\int_{y=x}^{N}(y-x)\,\mathrm{d}y. \tag{37.6}$$

解出来是 $\left(xy-\frac{1}{2}y^2\right)\,|_0^x+\left(\frac{1}{2}y^2-xy\right)\,|_x^N$，可以化简为 $\left(x^2-Nx+\frac{1}{2}N^2\right)$。现在我们要计算外层积分：

$$\int_{x=0}^{N}\left(x^2-Nx+\frac{1}{2}N^2\right)\mathrm{d}x, \tag{37.7}$$

结果是：

$$\left(\frac{1}{3}x^3-\frac{N}{2}x^2+\frac{N^2}{2}x\right)\,|_0^N=\frac{N^3}{3}. \tag{37.8}$$

别忘了，我们还要除以寻道的总数（N^2）才能得到平均寻道距离：$\left(\frac{N^3}{3}\right)/(N^2)=\frac{1}{3}N$。所以，在所有可能的寻道中，磁盘上的平均寻道距离是完整距离的三分之一。以后再听到平均寻道是完整寻道的三分之一，你就知道它是怎么来的了。

的那个。因此，磁盘调度程序在工作时会尽量遵循 **SJF（最短作业优先，shortest job first）原则**。

SSTF：最短寻道时间优先

一种早期的磁盘调度方法叫作**最短寻道时间优先**（shortest-seek-time-first，**SSTF**），也叫**最短寻道优先**（shortest-seek-first，**SSF**）。SSTF 按磁道对 I/O 请求队列（queue）排序，先选最近磁道上的请求来完成。例如，假设磁头当前位于内侧磁道上方，我们有两个请求，

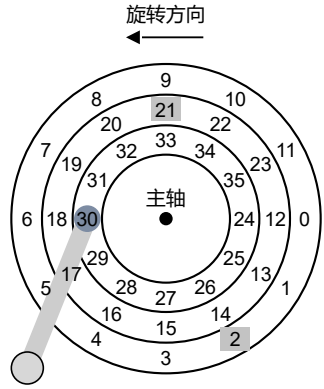


图 37.7：**SSTF：调度请求 21 和 2**

分别访问扇区 21（中间磁道）和扇区 2（外侧磁道），那么我们就先发出对 21 的请求，等它完成后，再发出对 2 的请求（图 37.7，第 9 页）。

在这个例子中，SSTF 先寻道到中间磁道，再寻道到外侧磁道，效果很好。然而，SSTF 并不是万灵药，原因如下。第一，主机操作系统看不到驱动器的几何结构；它看到的只是一个块数组。好在这个问题比较容易解决。操作系统可以不用 SSTF，而是简单地实现 **最近块优先**（nearest-block-first，**NBF**），即每次挑块地址（block address）最近的请求作为下一个来调度。

第二个问题更根本：**饥饿**（starvation）。在上面的例子中，设想有源源不断的请求访问磁头当前所在的内侧磁道。那么，纯粹的 SSTF 方法会完全忽略对其他任何磁道的请求。于是，问题的关键在于：

关键问题：如何处理磁盘饥饿
怎样才能实现类似 SSTF 的调度，同时避免饥饿?

电梯算法（又称 SCAN 或 C-SCAN）

这个问题的答案很早以前就有人提出来了（例如见 [CKR72]），而且相当直观。这个算法最初叫作 **SCAN**，它只是在磁盘上来回移动，按磁道的顺序依次处理请求。我们把在磁盘上的一趟移动——从外侧磁道到内侧磁道，或者从内侧到外侧——称为一次*扫描*（sweep）。因此，如果某个请求到来时，它要访问的磁道在这次扫描中已经扫过了，它不会马上得到处理，而是排队等下一次（反方向的）扫描。

SCAN 有许多变体，做的事情都差不多。例如，Coffman 等人提出了 **F-SCAN**，它在扫描期间冻结待处理的队列 [CKR72]；这样，扫描期间到达的请求会被放进另一个队列，留到以后处理。这样做推迟了处理晚到（但离得更近）的请求，从而避免了远处请求的饥饿。

C-SCAN 是另一种常见的变体，是 **Circular SCAN**（循环扫描）的简称。这个算法不在磁盘上双向扫描，而是只从外向内扫描，然后在外侧磁道复位，重新开始。这样对内侧和外侧磁道更公平一些，因为纯粹来回扫描的 SCAN 偏向中间的磁道：处理完外侧磁道后，SCAN 要经过中间两次，才会再回到外侧磁道。

SCAN 算法（及其近亲）有时被称为**电梯**（elevator）算法，原因现在应该很清楚了：它的行为就像电梯一样，要么往上走，要么往下走，而不是仅仅根据哪层楼更近来响应各层的请求。想象一下，如果你正从 10 楼下到 1 楼，有人在 3 楼进来按了 4 楼，结果电梯因为 4 楼比 1 楼“更近”就往上去了 4 楼，那该多烦人！可以看到，电梯算法（elevator algorithm）用在现实生活中，可以防止电梯里打起架来。用在磁盘上，它只是防止了饥饿。

遗憾的是，SCAN 及其近亲并不是最好的调度技术。具体来说，SCAN（甚至 SSTF）并没有尽可能严格地遵循 SJF 原则：它们忽略了旋转。于是，又一个关键问题来了：

关键问题：如何把磁盘旋转的开销考虑进去
怎样实现一个同时考虑寻道和旋转、从而更接近 SJF 的算法？

SPTF：最短定位时间优先

最短定位时间优先（shortest positioning time first，**SPTF**）调度（有时也叫 **最短访问时间优先**（shortest access time first，**SATF**））正是这个问题的解决办法。不过在讨论它之前，我们先把问题弄得更清楚一些。图 37.8 给出了一个例子。

在这个例子中，磁头当前位于内侧磁道的扇区 30 上方。于是，调度程序（scheduler）必须决定：下一个该调度中间磁道上的扇区 16，还是外侧磁道上的扇区 8？

答案当然是“看情况”。在工程中，“看情况”几乎总是正确答案，这反映出权衡取舍（trade-off）是工程师生活的一部分；这类格言在紧要关头也很好用，比如老板问了你一个答不上来的问题时，你不妨试试这句妙语。不过，知道为什么要看情况几乎总是更好，这正是我们这里要讨论的。

这里要看的是寻道时间和旋转时间的相对大小。在我们的例子中，如果寻道时间远大于旋转延迟，那么 SSTF（及其变体）就够用了。然而，设想寻道比旋转快不少。那么，在我们的例子中，寻道更远一些、去处理外侧磁道上的请求 8，会比寻道较短的距离到中间磁道去处理请求 16 更划算，因为扇区 16 要转上几乎一整圈才会经过磁头下方。

正如上面看到的，在现代驱动器上，寻道和旋转大致相当（当然，这要看具体的请求），因此 SPTF 很有用，可以提高性能。然而，在操作系统中实现它就更难了，因为操作系统通常不太清楚磁道的边界在哪里，也不知道磁头当前（在旋转意义上）在哪里。因此，SPTF 通常是在驱动器内部实现的，下面会介绍。

其他调度问题

关于基本磁盘操作、调度及相关话题，这个简短的介绍还有许多问题没有讨论。其中一个问题是：在现代系统中，磁盘调度是在哪里进行的？在较老的系统中，所有调度都由操

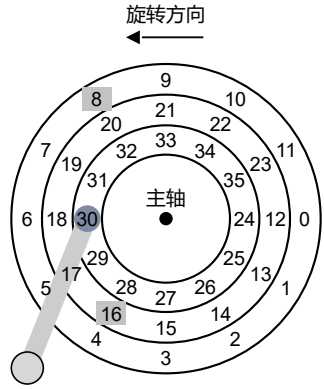


图 37.8：**SSTF：有时还不够好**

> 提示：答案总是“看情况”（Livny 定律）
> 几乎任何问题都可以用“看情况”来回答，我们的同事 Miron Livny 总是这么说。不过，用的时候要小心，因为如果你用这种方式回答太多问题，人们就再也不会问你问题了。例如，有人问：“想去吃午饭吗?”你回答：“看情况，你去吗?”

作系统来做；操作系统检查一遍待处理的请求，挑出最好的一个，把它发给磁盘。那个请求完成后，再选出下一个，依此类推。那时候磁盘更简单，生活也更简单。

在现代系统中，磁盘可以同时容纳多个尚未完成的请求，而且自身就有精巧的内部调度程序——它们可以准确地实现 SPTF，因为在磁盘控制器内部，所有相关细节都能拿到，包括磁头的确切位置。因此，操作系统的调度程序通常会挑出它认为最好的几个请求（比如 16 个），把它们全部发给磁盘；磁盘再利用它掌握的磁头位置和详细的磁道布局信息，以尽可能好的（SPTF）顺序处理这些请求。

磁盘调度程序执行的另一项重要相关任务是 **I/O 合并**（I/O merging）。例如，设想有一串请求，依次读取块 33、块 8、块 34，就像图 37.8 中那样。在这种情况下，调度程序应该把对块 33 和块 34 的请求**合并**（merge）成一个读两个块的请求；调度程序所做的任何重新排序，都是在合并后的请求上进行的。合并在操作系统层面尤其重要，因为它减少了发给磁盘的请求数，从而降低了开销。

现代调度程序要解决的最后一个问题是：系统在向磁盘发出 I/O 之前，应该等待多长时间？有人可能会天真地认为，磁盘只要有一个 I/O，就应该立即把请求发给驱动器；这种方法称为 **work-conserving**，因为只要有请求要处理，磁盘就永远不会空闲。然而，关于**预期磁盘调度**（anticipatory disk scheduling）的研究表明，有时候稍等一会儿反而更好 [ID01]，这就是所谓的 **non-work-conserving** 方法。等一等，可能会有一个新的、“更好”的请求到达磁盘，从而提高整体效率。当然，决定什么时候等、等多久，可能很棘手；详见那篇研究

论文，或者（如果你是个有抱负的人）去看看 Linux 内核的实现，看看这样的思想是怎样落地到实践中的。

37.6 小结

我们概述了磁盘是怎样工作的。这个概述实际上是一个详细的功能模型；它并没有描述实际驱动器设计中那些令人惊叹的物理学、电子学和材料科学。对这方面更多细节感兴趣的人，我们建议换一个专业（或者辅修一个）；对这个模型感到满意的人，很好！现在我们可以利用这个模型，在这些不可思议的设备之上构建更有意思的系统了。

 © 2008–23, ARPACI-DUSSEAU

参考文献

[ADR03] "More Than an Interface: SCSI vs. ATA" by Dave Anderson, Jim Dykes, Erik Riedel. FAST '03, 2003. *One of the best recent-ish references on how modern disk drives really work; a must read for anyone interested in knowing more.*

[CKR72] "Analysis of Scanning Policies for Reducing Disk Seek Times" E.G. Coffman, L.A. Klimko, B. Ryan SIAM Journal of Computing, September 1972, Vol 1. No 3. *Some of the early work in the field of disk scheduling.*

[HK+17] "The Unwritten Contract of Solid State Drives" by Jun He, Sudarsun Kannan, Andrea C. Arpaci-Dusseau, Remzi H. Arpaci-Dusseau. EuroSys '17, Belgrade, Serbia, April 2017. *We take the idea of the unwritten contract, and extend it to SSDs. Using SSDs well seems as complicated as hard drives, and sometimes more so.*

[ID01] "Anticipatory Scheduling: A Disk-scheduling Framework To Overcome Deceptive Idleness In Synchronous I/O" by Sitaram Iyer, Peter Druschel. SOSP '01, October 2001. *A cool paper showing how waiting can improve disk scheduling: better requests may be on their way!*

[JW91] "Disk Scheduling Algorithms Based On Rotational Position" by D. Jacobson, J. Wilkes. Technical Report HPL-CSP-91-7rev1, Hewlett-Packard, February 1991. *A more modern take on disk scheduling. It remains a technical report (and not a published paper) because the authors were scooped by Seltzer et al. [SCO90].*

[RW92] "An Introduction to Disk Drive Modeling" by C. Ruemmler, J. Wilkes. IEEE Computer, 27:3, March 1994. *A terrific introduction to the basics of disk operation. Some pieces are out of date, but most of the basics remain.*

[SCO90] "Disk Scheduling Revisited" by Margo Seltzer, Peter Chen, John Ousterhout. USENIX 1990. *A paper that talks about how rotation matters too in the world of disk scheduling.*

[SG04] "MEMS-based storage devices and standard disk interfaces: A square peg in a round hole?" Steven W. Schlosser, Gregory R. Ganger FAST '04, pp. 87-100, 2004 *While the MEMS aspect of this paper hasn't yet made an impact, the discussion of the contract between file systems and disks is wonderful and a lasting contribution. We later build on this work to study the "Unwritten Contract of Solid State Drives" [HK+17]*

[S09a] "Barracuda ES.2 data sheet" by Seagate, Inc.. Available at this website, at least, it was: http://www.seagate.com/docs/pdf/datasheet/disc/ds_barracuda_es.pdf. *A data sheet; read at your own risk. Risk of what? Boredom.*

[S09b] "Cheetah 15K.5" by Seagate, Inc.. Available at this website, we're pretty sure it is: http://www.seagate.com/docs/pdf/datasheet/disc/ds-cheetah-15k-5-us.pdf. *See above commentary on data sheets.*

作业（模拟）

这个作业用 `disk.py` 让你熟悉现代硬盘驱动器是怎样工作的。它有很多不同的选项，而且和其他大多数模拟程序不同，它有一个图形动画，可以让你确切地看到磁盘工作时发生了什么。详情请见 README。

1. 计算下面几组请求的寻道时间、旋转时间和传输时间（transfer time）：`-a 0`、`-a 6`、`-a 30`、`-a 7,30,8`，最后是 `-a 10,11,12,13`。
2. 还是上面这些请求，但把寻道速率改成不同的值：`-S 2`、`-S 4`、`-S 8`、`-S 10`、`-S 40`、`-S 0.1`。时间会怎样变化？
3. 还是上面这些请求，但改变旋转速率：`-R 0.1`、`-R 0.5`、`-R 0.01`。时间会怎样变化？
4. FIFO 并不总是最好的。例如，对于请求流 `-a 7,30,8`，应该按什么顺序处理这些请求？在这个工作负载上运行最短寻道时间优先（SSTF）调度程序（`-p SSTF`）；处理每个请求应该花多长时间，寻道、旋转、传输各要多少？
5. 现在使用 shortest access-time first（SATF）调度程序（`-p SATF`）。对于 `-a 7,30,8` 这个工作负载，它有什么不同吗？找出一组 SATF 比 SSTF 表现更好的请求；更一般地说，什么时候 SATF 比 SSTF 好？
6. 来试试这个请求流：`-a 10,11,12,13`。运行时哪里表现不好？试着加入磁道偏斜（`-o skew`）来解决这个问题。在默认寻道速率下，偏斜应该设成多少才能让性能最好？换成不同的寻道速率（例如 `-S 2`、`-S 4`）呢？一般来说，你能写出一个计算偏斜的公式吗？
7. 指定一个每个区密度不同的磁盘，例如 `-z 10,20,30`，它指定了外侧、中间和内侧磁道上相邻块之间的角度差。运行一些随机请求（例如 `-a -1 -A 5,-1,0`，它通过 `-a -1` 选项指定使用随机请求，并生成 5 个范围从 0 到最大值的请求），计算寻道时间、旋转时间和传输时间（transfer time）。使用不同的随机种子。外侧、中间和内侧磁道的带宽（bandwidth，以每单位时间的扇区数计）分别是多少？
8. 调度窗口决定了磁盘一次能考察多少个请求。生成随机工作负载（使用不同的随机种子，例如 `-A 1000,-1,0`），看看调度窗口从 1 增大到请求数时，SATF 调度程序要花多长时间。窗口要多大才能让性能最好？提示：用 `-c` 选项、不开图形（`-G`），这样跑得更快。调度窗口设为 1 时，用哪种策略还有关系吗？
9. 假设采用 SATF 策略，构造一串请求，让某个特定的请求饥饿。对于这串请求，如果使用 **bounded SATF**（**BSATF**）调度方法，表现如何？在这种方法中，你需要指定调度窗口（例如 `-w 4`）；只有当前窗口里的所有请求都处理完之后，调度程序才会移到下一个请求窗口。这样能解决饥饿问题吗？与 SATF 相比，它的表现如何？一般来说，磁盘应该怎样在性能和避免饥饿之间做权衡？
10. 到目前为止，我们看过的所有调度策略（scheduling policy）都是**贪心**（greedy）的：它们只挑眼下最好的选项，而不去寻找最优的调度。你能找出一组让贪心策略不是最优的请求吗？

 © 2008–23, ARPACI-DUSSEAU